Project Report Format

1. **INTRODUCTION**
   - 1.1 Project Overview
   - 1.2 Purpose
2. **IDEATION PHASE**
   - 2.1 Problem Statement
   - 2.2 Empathy Map Canvas
   - 2.3 Brainstorming
3. **REQUIREMENT ANALYSIS**
   - 3.1 Customer Journey map
   - 3.2 Solution Requirement
   - 3.3 Data Flow Diagram
   - 3.4 Technology Stack
4. **PROJECT DESIGN**
   - 4.1 Problem Solution Fit
   - 4.2 Proposed Solution
   - 4.3 Solution Architecture
5. **PROJECT PLANNING & SCHEDULING**
   - 5.1 Project Planning
6. **FUNCTIONAL AND PERFORMANCE TESTING**
   - 6.1 Performance Testing
7. **RESULTS**
   - 7.1 Output Screenshots
8. **ADVANTAGES & DISADVANTAGES**
9. **CONCLUSION**
10. **FUTURE SCOPE**
11. **APPENDIX**

    Source Code(if any)

    Dataset Link

    GitHub & Project Demo Link

PROJECT REPORT

Script-Controlled ACL – Restrict Record Access Based on Field Value

Team ID: 6ab8c2c230f9767f87730236

Report Date: 04 October 2026

Final consolidated report prepared from the supplied project templates, planning records, solution design, and UAT/performance-testing documents.

Report structure follows the supplied Project Report Format.

1. INTRODUCTION

1.1 Project Overview

The project addresses record-level access control where visibility should depend on a value stored in the record. The supplied design identifies a limitation in relying only on static roles or groups: different records may require different visibility decisions. The proposed approach uses a script-controlled ACL that evaluates a designated record field together with the user's access context and then permits or denies access.

1.2 Purpose

• Provide fine-grained, dynamic record access based on business rules.

• Prevent unauthorized users from viewing restricted records.

• Reduce repeated manual changes to static access-control rules.

• Make access decisions predictable and auditable at record level.

• Ensure access can change automatically when the controlling field changes.

Project identity

Item	Value
Team ID	6ab8c2c230f9767f87730236
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Primary outcome	Dynamic allow/deny decision for each record based on a controlling field value
Testing status	Supplied functional/performance and UAT records report passing test scenarios

2. IDEATION PHASE

2.1 Problem Statement

Traditional ACL implementations may depend heavily on static roles or groups and may not provide sufficient record-level control when access depends on values inside individual records. This can create unnecessary exposure risk, administrative effort, and uncertainty about whether the correct users can see each record.

2.2 Empathy Map Canvas

Dimension	Key observations
Users / stakeholders	Application users, record owners, managers, administrators, and teams handling sensitive or business-specific data.
Think / feel	Need confidence that relevant records are visible while restricted records remain protected.
See	Different records carrying different field values and different access expectations.
Hear	Business rules and security requirements that may change as record values change.
Pain points	Manual checks, static permissions, repeated ACL maintenance, and risk of overexposure.
Needs / gains	Automatic, predictable, field-aware access decisions and easier policy maintenance.

2.3 Brainstorming

• Use condition-based access rules rather than only fixed role/group permissions.

• Evaluate the controlling field at the time the record is accessed.

• Combine field-value evaluation with the user's role/access context.

• Deny access for restricted, missing, or unsupported values when the business rule requires it.

• Apply the same policy consistently across normal record access and tested direct access paths.

• Keep the rule reusable so that different fields and business conditions can be supported.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User action / experience	System expectation
1. Record request	User opens, searches for, or queries a record.	Identify the record and current access context.
2. Context evaluation	System identifies the user's role/access context.	Use the current user context consistently.
3. Field evaluation	System reads the controlling field.	Evaluate the field value against the configured business condition.
4. Decision	Access is granted or denied.	Return the configured allow/deny outcome.
5. Outcome	User sees permitted data or is prevented from viewing restricted data.	No restricted data path should bypass the server-side rule.

3.2 Solution Requirement

• Define the target record and the controlling field used for the access decision.

• Define the mapping between field values and permitted users/roles.

• Execute the ACL logic for each access attempt.

• Support both positive and negative conditions, including restricted or unsupported values.

• Validate behavior for authorized and unauthorized users.

• Verify that the rule remains consistent across repeated and mixed-record tests.

• Document the configuration, test evidence, and deployment verification.

3.3 Data Flow Diagram

Flow	Description
User → Access request	A user requests a record through the application.
Access request → ACL	The request reaches the access-control layer.
ACL → Record field	The ACL reads the designated controlling field.
ACL → User context	The ACL considers the user's authorization context.
Decision → Record	The rule returns allow or deny.
Decision → Audit / result	The access outcome is reflected in the application's behavior and can be reviewed during security testing.

3.4 Technology Stack

The supplied project documents do not name a specific vendor/platform stack. To avoid inventing unsupported platform details, the stack is described functionally below.

Layer	Functional technology/component
Access-control layer	Script-controlled ACL / server-side authorization logic
Data layer	Application records containing the designated controlling field
Identity layer	User and role/access context used by the authorization decision
Application layer	Record navigation, search, query, and direct record-access paths
Testing layer	Functional, performance, repeated-access, mixed-record, and UAT validation
Governance	Configuration documentation, defect/status reporting, and version-control evidence

4. PROJECT DESIGN

4.1 Problem Solution Fit

The supplied Problem–Solution Fit document identifies the core fit as follows: users need different record visibility; access rules may depend on changing field values; static permissions may be insufficient; and administrators need predictable, auditable decisions. The proposed script-controlled ACL directly addresses these conditions by reading the current record field and applying the configured business rule.

4.2 Proposed Solution

Parameter	Completed solution
Problem Statement	Static roles/groups may not support record-level access based on a field value, creating risk of unauthorized
Idea / Solution	Use a script-controlled ACL that evaluates a specified record field and the user's permissions before allowing
Novelty / Uniqueness	Fine-grained dynamic access control combining ACL rules with field-value evaluation, reducing dependence
Social Impact / Customer Satisfaction	Improves data privacy and security, reduces accidental exposure, and increases user confidence.
Business Model	Potential enterprise security module through subscription/licensing plus implementation, customization, main
Scalability	Reusable across applications, tables, departments, and organizations with configurable field-based rules.

4.3 Solution Architecture

Functional architecture derived from the supplied solution description:

Component	Responsibility
User / Client	Initiates record access.
Application / Record Service	Receives the request and identifies the target record.
Authorization Context	Provides user/role information needed for the decision.
Script-Controlled ACL	Reads the controlling field, applies the business condition, and returns allow/deny.
Record Store	Holds the records and their controlling field values.
Result / Audit	Enforces the decision and provides evidence for testing and review.

End-to-end flow: User request → Application → ACL → read field + user context → business rule → Allow/Deny → enforced record access.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The completed planning template defines two project sprints focused on access requirements, ACL design, implementation, testing, and deployment.

Sprint	User stories / tasks	Story points	Priority
Sprint-1	USN-1 requirements analysis; USN-2 access matrix; USN-3 scripted ACL design; USN-4 ACL configuration structure	16	High
Sprint-2	USN-5 ACL implementation; USN-6 authorization testing; USN-7 edge-case testing; USN-8 deployment documentation	16	High/Medium

Sprint schedule from the supplied planning record

Sprint	Duration	Start	End	Completed
Sprint-1	6 days	17 Feb 2025	22 Feb 2025	16 story points
Sprint-2	6 days	24 Feb 2025	01 Mar 2025	16 story points

Velocity calculation

Total completed story points = 16 + 16 = 32. Average team velocity = 32 / 2 = 16 story points per sprint. The planning document also records a planned delivery rate of 16 / 6 = 2.67 story points per day and 0.375 day per story point.

Planning note: the source planning template contains example material as well as the completed project plan; the project-specific values above are taken from the completed entries.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The completed functional/performance testing record reports all listed scenarios as Pass.

Test ID	Scenario	Result
FT-01	ACL field-value validation for allowed, restricted, blank, and unsupported values	Pass
FT-02	User/role authorization validation	Pass
FT-03	Record access enforcement through tested access paths	Pass
FT-04	ACL script execution check	Pass
PT-01	Access decision response time / normal-use degradation	Pass
PT-02	Repeated access/load consistency	Pass
PT-03	Multiple-record independent evaluation	Pass

UAT summary

UAT section	Total	Not tested	Fail	Pass
Record Access Validation	8	0	0	8
Client / Role Access	6	0	0	6
Security Controls	6	0	0	6
Field Value Conditions	6	0	0	6
Total	26	0	0	26

Defect status: the completed UAT report records zero unresolved defects across the listed resolution categories at UAT completion.

7. RESULTS

7.1 Output Screenshots

The following figures reproduce selected pages from the completed project artifacts supplied for this report. They are included as evidence of the completed design, planning, and UAT work.

Project Design Phase
Problem – Solution Fit Template

Date	15 February 2025
Team ID	~~SWTID1741977730236~~
Project Name	
Maximum Marks	2 Marks

Problem – Solution Fit Template:

The Problem-Solution Fit simply means that you have found a problem with your customer and that the solution you have realized for it actually solves the customer's problem. It helps entrepreneurs, marketers and corporate innovators identify behavioral patterns and recognize what would work and why

Purpose:

- ❑ Solve complex problems in a way that fits the state of your customers.
- ❑ Succeed faster and increase your solution adoption by tapping into existing mediums and channels of behavior.
- ❑ Sharpen your communication and marketing strategy with the right triggers and messaging.
- ❑ Increase touch-points with your company by finding the right problem-behavior fit and building trust by solving frequent annoyances, or urgent or costly problems.
- ❑ **Understand the existing situation in order to improve it for your target group.**

Template:



References:

1. https://www.ideahackers.network/problem-solution-fit-canvas/
2. https://medium.com/@epicantus/problem-solution-fit-canvas-aa3dd59cb4fe

Figure 1. Completed Problem–Solution Fit artifact.

Project Design Phase
Proposed Solution Template

Date	15 February 2025
Team ID	6ab8c2c230f9767f87730236
Project Name	Script-Controlled ACL - Restrict Record Access Based on Field Value
Maximum Marks	2 Marks

Proposed Solution Template:

Project team shall fill the following information in the proposed solution template.

S.No.	Parameter	Description
1.	Problem Statement (Problem to be solved)	Traditional ACLs may rely on static roles or groups and may not support record-level access based on a field value. This can expose sensitive records to users who should not have access.
2.	Idea / Solution description	Implement a script-controlled ACL that evaluates a specified record field and the user's permissions before allowing access. The script dynamically grants or denies access for each record.
3.	Novelty / Uniqueness	Provides fine-grained, dynamic access control by combining ACL rules with field-value evaluation, reducing the need for many separate roles or static ACL rules.
4.	Social Impact / Customer Satisfaction	Improves data privacy and security, reduces accidental exposure of sensitive records, and increases user confidence in the system's access-control process.
5.	Business Model (Revenue Model)	Can be offered as an enterprise security module through subscription or licensing, with additional revenue from implementation, customization, maintenance, and support services.
6.	Scalability of the Solution	The approach can be reused across applications, tables, departments, and organizations. Field-based rules can be configured for different business requirements and scaled for larger enterprise deployments.

Figure 2. Completed Proposed Solution artifact.

Additional evidence

Project Planning Phase

Project Planning Template (Product Backlog, Sprint Planning, Stories, Story points)

Date	15 February 2025
Team ID	6ab8c2c230f9767f87730236
Project Name	Script-Controlled ACL ? Restrict Record Access Based on Field Value
Maximum Marks	5 Marks

Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Use the below template to create product backlog and sprint schedule

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1	Access Requirements	USN-1	As a project team, I can analyze field-based record access requirements and identify who should be allowed or denied.	5	High	Team 6
Sprint-1	Access Requirements	USN-2	As a project team, I can define an access matrix mapping field values to permitted users or roles.	5	High	Team 6
Sprint-1	ACL Design	USN-3	As an administrator, I can design scripted ACL logic that evaluates the target field value before granting record access.	3	High	Team 6
Sprint-1	ACL Design	USN-4	As an administrator, I can configure the ACL target, field conditions, and supporting script structure.	3	High	Team 6
Sprint-2	ACL Implementation	USN-5	As an authorized user, I can access a record only when its field value satisfies the scripted ACL rule.	5	High	Team 6
Sprint-2	Testing	USN-6	As a tester, I can verify that permitted users receive access and unauthorized users are denied access.	3	High	Team 6
Sprint-2	Testing	USN-7	As a tester, I can validate edge cases such as missing or unexpected field values and role interactions.	3	Medium	Team 6
Sprint-2	Deployment	USN-8	As a project team, I can document, deploy, and verify the scripted ACL configuration in the target environment.	5	Medium	Team 6

Figure 3. Completed project planning and sprint artifact.

Acceptance Testing
UAT Execution & Report Submission

Date	04 October 2026
Team ID	6ab8c2c230f9767f87730236
Project Name	Script-Controlled ACL - Restrict Record Access Based on Field Value
Maximum Marks	4 Marks

1. Purpose of Document

The purpose of this document is to briefly explain the UAT test coverage, results, and open issues for the Script-Controlled ACL – Restrict Record Access Based on Field Value project at release to UAT.

2. Defect Analysis

This report shows the number of UAT defects by severity and their resolution status.
At UAT completion, no unresolved defects remain.

Resolution	Severity 1	Severity 2	Severity 3	Severity 4	Subtotal
By Design	0	0	0	0	0
Duplicate	0	0	0	0	0
External	0	0	0	0	0
Fixed	0	0	0	0	0
Not Reproduced	0	0	0	0	0
Skipped	0	0	0	0	0
Won't Fix	0	0	0	0	0
Totals	0	0	0	0	0

3. Test Case Analysis

This report shows the number of test cases that have passed, failed, and remain untested.

Section	Total Cases	Not Tested	Fail	Pass
Record Access Validation	8	0	0	8
Client / Role Access	6	0	0	6
Security Controls	6	0	0	6
Field Value Conditions	6	0	0	6

Figure 4. Completed UAT execution/report artifact.

Overall result

- The proposed solution provides dynamic record-level access decisions using a controlling field.
- Functional tests FT-01 through FT-04 passed.
- Performance-oriented tests PT-01 through PT-03 passed.
- The UAT summary records 26/26 test cases passed with no failed or untested cases.
- The supplied UAT report records no unresolved defects at completion.

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages / considerations
Fine-grained record-level control.	Script logic must be carefully designed and maintained.
Access can change automatically with field values.	Incorrect conditions can deny legitimate access or expose data.
Reduces the need for many static ACL variations.	Testing must cover positive, negative, blank, and unexpected values.
Improves privacy and reduces accidental exposure.	Complex business rules may increase configuration complexity.
Reusable across different business conditions.	Performance should be monitored as rule complexity and request volume grow.

9. CONCLUSION

The project demonstrates a practical approach to dynamic record-level authorization. Instead of treating access as a fixed permission only, the solution evaluates a designated field in the current record together with the user's authorization context. The completed design and testing artifacts indicate that the intended allow/deny behavior was achieved for the tested scenarios. The UAT summary reports 26 passing cases, with no failures, no untested cases, and no unresolved defects recorded at completion.

The main value of the solution is consistent enforcement of business-specific access rules while reducing repetitive manual ACL changes. Continued governance, testing, and careful rule maintenance remain important for production use.

10. FUTURE SCOPE

- Extend the approach to multiple controlling fields and compound business conditions.
- Provide reusable rule templates for common record-security patterns.
- Add richer audit reporting for allow/deny decisions and policy changes.
- Introduce automated regression testing for ACL rule changes.
- Add monitoring for unusual access-denial or access-attempt patterns.
- Support centralized policy administration across multiple applications or organizational units.
- Evaluate performance at larger record volumes and higher concurrent request loads.

11. APPENDIX

Source Code (if any)

No separate source-code repository or code listing was included in the supplied final-report materials. The implementation approach is documented through the proposed solution and functional/performance test artifacts.

Dataset Link

Not provided in the supplied project materials.

GitHub & Project Demo Link

Not provided in the supplied project materials. No external link has been invented.

Referenced project artifacts

• Problem-Solution_Fit_Script-Controlled_ACL_Completed.pdf

• Proposed_Solution_Template_Final.pdf

• Solution_Architecture_Completed.pdf

• Project_Planning_Template_Completed.pdf

• UAT_Report_Script_Controlled_ACL_Final.pdf

• 06_Functional_Performance_Testing_Completed.docx

End of Final Report